

Nama: Robani Diansyah

NPM: 23312246

Kelas: IF 23 Fx

A. Konfigurasi Database

schema.prisma

```
// This is your Prisma schema file,
// learn more about it in the docs: https://pris.ly/d/prisma-schema

// Looking for ways to speed up your queries, or scale easily with
// your serverless or edge functions?
// Try Prisma Accelerate: https://pris.ly/cli/accelerate-init

generator client {
  provider = "prisma-client-js"
  engineType = "library"
}

datasource db {
  provider = "postgresql"
  url = env("DATABASE_URL")
}

// User Model - Untuk autentikasi dan job posting
model User {
  id          String   @id @default(cuid())
  email       String   @unique
  password    String
  name        String
  role        Role     @default(JOBSEEKER)
  jobs        Job[]
  createdAt   DateTime @default(now())
  updatedAt   DateTime @updatedAt

  @@map("users")
}

// Job Model - untuk list pekerjaan
model Job {
  id          String   @id @default(cuid())
  title       String
  company     String
  location    String
  type        JobType  @default(FULLTIME)
  salary      String?
```

```

description String @db.Text
requirements String @db.Text
userId String
user User @relation(fields: [userId], references: [id],
onDelete: Cascade)
createdAt DateTime @default(now())
updatedAt DateTime @updatedAt

@@index([userId])
@@map("jobs")
}

// Enum untuk role user
enum Role {
  JOBSEEKER
  EMPLOYER
  ADMIN
}

// Enum untuk tipe pekerjaan
enum JobType {
  FULLTIME
  PARTTIME
  CONTRACT
  FREELANCE
  INTERNSHIP
}

```

Penjelasan Alur Database: Kode di atas mendefinisikan struktur database (Schema) menggunakan Prisma ORM dengan database PostgreSQL. Berikut poin-poin utamanya:

1. Model User:

- Tabel ini menyimpan data otentikasi pengguna (email, password) dan profil dasar.
- Terdapat kolom role yang menggunakan Enum, sehingga tipe akun terkunci hanya boleh: JOBSEEKER (Pelamar), EMPLOYER (Penyedia Kerja), atau ADMIN.

2. Model Job:

- Tabel ini menyimpan data lowongan pekerjaan.
- Menggunakan tipe data @db.Text untuk description dan requirements agar bisa menampung teks panjang.
- Terdapat relasi One-to-Many dengan User (field user). Artinya, satu User (perusahaan) bisa memposting banyak lowongan.
- Fitur onDelete: Cascade memastikan jika akun perusahaan dihapus, semua lowongan buatannya otomatis ikut terhapus agar database tetap bersih.

3. Enums:

- Digunakan untuk menjaga konsistensi data pada kolom Role dan JobType (Fulltime, Freelance, dll), meminimalisir kesalahan input data (typo).

B. Konfigurasi dan Keamanan Sistem

Koneksi Database

Prisma.ts

```
import { PrismaClient } from "@prisma/client";

// Mencegah error "Too many connections" saat hot-reload (Development)
const globalForPrisma = globalThis as unknown as {
  prisma: PrismaClient | undefined;
}

// Cek global var: Pakai koneksi lama jika ada, atau buat baru (Singleton)
export const prisma =
  globalForPrisma.prisma ??
  new PrismaClient({
    log:
      process.env.NODE_ENV === "development"
        ? ["query", "error", "warn"]
        : ["error"],
  });

// Simpan koneksi ke memori global jika sedang coding (bukan production)
if (process.env.NODE_ENV !== "production") {
  globalForPrisma.prisma = prisma;
}

// Helper untuk memutus koneksi database manual
export async function disconnectPrisma(): Promise<void> {
  await prisma.$disconnect();
}
```

Penjelasan Alur Kode: File ini mengatur inisialisasi koneksi ke database menggunakan Singleton Pattern.

- **Mencegah Koneksi Bocor:** Saat tahap development, Next.js sering melakukan reload otomatis. Kode `globalForPrisma` memastikan aplikasi tidak membuat koneksi baru berulang-ulang yang bisa menyebabkan error "Too many connections".
- **Log Query:** Sistem akan menampilkan log SQL di terminal hanya saat mode development untuk memudahkan debugging, namun mematikannya di production agar performa lebih cepat.

Helper Enkripsi Password

Auth.ts

```
import bcrypt from "bcrypt";
import { jwtVerify } from "jose";
```

```

import { NextRequest, NextResponse } from "next/server";

const JWT_SECRET = process.env.JWT_SECRET || "ini-adalah-my-kunci-guweh";

// 1. Hash Password
export async function hashPassword(password: string): Promise<string> {
  return await bcrypt.hash(password, 10);
}

// 2. Cek Password
export async function comparePassword(plain: string, hashed: string):
Promise<boolean> {
  return await bcrypt.compare(plain, hashed);
}

// 3. Helper Validasi Token (Auth Middleware)
export async function authenticateRequest(request: NextRequest) {
  const authHeader = request.headers.get("Authorization");

  if (!authHeader || !authHeader.startsWith("Bearer ")) {
    return { error: NextResponse.json({ error: "Unauthorized" }, { status:
401 }) };
  }

  const token = authHeader.split(" ")[1];

  try {
    const secret = new TextEncoder().encode(JWT_SECRET);
    const { payload } = await jwtVerify(token, secret);

    // Kembalikan data user dari token
    return { user: payload as { userId: string, role: string } };
  } catch (error) {
    console.error("Auth error:", error);
    return { error: NextResponse.json({ error: "Token Invalid" }, { status:
401 }) };
  }
}

```

Penjelasan Alur Kode: File ini berisi fungsi utilitas untuk keamanan data sensitif pengguna (password).

- **Keamanan Password:** Fungsi `hashPassword` dan `comparePassword` menggunakan algoritma *bcrypt* untuk mengacak dan memverifikasi password user.
- **Validasi Token (Middleware):** Fungsi `authenticateRequest` bertindak sebagai validator sesi. Ia mengecek keberadaan Header Authorization pada request, lalu memverifikasi keaslian Token JWT menggunakan library *jose*. Jika token valid,

fungsi mengembalikan data user (ID dan Role) untuk diproses lebih lanjut oleh controller.

Konfigurasi CORS

Middleware.ts

```
import { NextResponse } from "next/server";
import type { NextRequest } from "next/server";

export function middleware(req: NextRequest) {
  // Handle preflight CORS
  if (req.method === "OPTIONS") {
    return new NextResponse(null, {
      status: 204,
      headers: {
        "Access-Control-Allow-Origin": "http://localhost:3001",
        "Access-Control-Allow-Methods": "GET, POST, PUT, PATCH, DELETE, OPTIONS",
        "Access-Control-Allow-Headers": "Content-Type",
      },
    });
  }

  const res = NextResponse.next();

  res.headers.set("Access-Control-Allow-Origin", "http://localhost:3001");
  res.headers.set("Access-Control-Allow-Methods", "GET, POST, PUT, PATCH, DELETE, OPTIONS");
  res.headers.set("Access-Control-Allow-Headers", "Content-Type");

  return res;
}

export const config = {
  matcher: "/api/:path*",
};
```

Penjelasan Alur Kode: Middleware ini berfungsi sebagai gerbang utama yang mengatur izin akses antar-aplikasi (Cross-Origin Resource Sharing).

- **Preflight Request:** Kode `if (req.method === "OPTIONS")` menangani permintaan awal browser untuk memastikan server aman diakses.
- **Access Control:**
 - **Allow-Origin:** Mengunci akses API hanya dari alamat Frontend (`http://localhost:3001`), memblokir akses dari website asing.

- Allow-Methods: Mengizinkan metode HTTP lengkap (GET, POST, PUT, DELETE) agar Frontend bisa melakukan operasi CRUD.

C. Implementasi Modul Autentikasi

Endpoint Registrasi

register/route.ts

```
import { NextRequest, NextResponse } from "next/server";
import { prisma } from "@/lib/prisma";
import bcrypt from "bcrypt";

export async function POST(req: NextRequest) {
  try {
    const { name, email, password, role } = await req.json();

    if (!name || !email || !password || !role) {
      return NextResponse.json({ error: "Semua field wajib diisi" }, {
status: 400 });
    }

    // cek email
    const exists = await prisma.user.findUnique({ where: { email } });
    if (exists) {
      return NextResponse.json({ error: "Email sudah terdaftar" }, { status:
400 });
    }

    // hash password
    const hashedPassword = await bcrypt.hash(password, 10);

    // simpan user
    const user = await prisma.user.create({
      data: { name, email, password: hashedPassword, role },
    });

    return NextResponse.json({ user: { id: user.id, name: user.name, email:
user.email, role: user.role } });
  } catch (err) {
    console.error(err);
    return NextResponse.json({ error: "Gagal register" }, { status: 500 });
  }
}
```

Penjelasan Alur Kode: Endpoint ini berfungsi untuk mendaftarkan pengguna baru ke dalam sistem. Alur logikanya adalah:

1. **Validasi Input:** Sistem mengecek kelengkapan data (name, email, password, role). Jika ada yang kosong, API mengembalikan status 400 Bad Request.
2. **Cek Duplikasi:** Menggunakan `prisma.user.findUnique` untuk memastikan email belum pernah terdaftar sebelumnya.
3. **Enkripsi Password:** Sebelum disimpan, password diacak menggunakan fungsi `bcrypt.hash` dengan cost factor 10. Ini standar keamanan agar password tidak terbaca jika database diretas.
4. **Penyimpanan Data:** Data user baru disimpan ke database, dan API mengembalikan data profil (tanpa password) sebagai konfirmasi sukses.

Endpoint Login

login/route.ts

```
import { NextRequest, NextResponse } from "next/server";
import { prisma } from "@/lib/prisma";
import { comparePassword } from "@/lib/auth";
import { SignJWT } from "jose";

const corsHeaders = {
  "Access-Control-Allow-Origin": "http://localhost:3001",
  "Access-Control-Allow-Methods": "POST, OPTIONS",
  "Access-Control-Allow-Headers": "Content-Type",
};

export async function OPTIONS() {
  return NextResponse.json({}, { headers: corsHeaders });
}

export async function POST(request: NextRequest) {
  try {
    const body = await request.json();
    const { email, password } = body;

    const user = await prisma.user.findUnique({ where: { email } });

    if (!user) {
      return NextResponse.json({ error: "Email tidak terdaftar" }, { status: 404, headers: corsHeaders });
    }

    const isValid = await comparePassword(password, user.password);
    if (!isValid) {
      return NextResponse.json({ error: "Password salah" }, { status: 401, headers: corsHeaders });
    }
  }
}
```

```

// Buat Token JWT
const secret = new TextEncoder().encode(process.env.JWT_SECRET ||
"rahasia-negara");
const token = await new SignJWT({ userId: user.id, role: user.role })
  .setProtectedHeader({ alg: "HS256" })
  .setExpirationTime("7d")
  .sign(secret);

// Buat object baru agar tidak perlu destructuring user.password yang
unused
const userData = {
  id: user.id,
  name: user.name,
  email: user.email,
  role: user.role
};

return NextResponse.json(
  { message: "Login berhasil", token, user: userData },
  { status: 200, headers: corsHeaders }
);
} catch (error) {
  console.error("Login error:", error);
  return NextResponse.json({ error: "Server error" }, { status: 500,
headers: corsHeaders });
}
}

```

Penjelasan Alur Kode: Endpoint ini menangani proses otentikasi dan pembuatan token sesi.

1. **Verifikasi User:** Sistem mencari email user di database. Jika ditemukan, password yang diinput akan dicocokkan dengan password terenkripsi di database menggunakan `comparePassword`.
2. **Pembuatan Token (JWT):** Jika kredensial valid, sistem menggunakan library `jose` (`SignJWT`) untuk membuat token akses. Token ini berisi payload penting (`userId` dan `role`), ditandatangani dengan algoritma `HS256`, dan diset kedaluwarsa dalam 7 hari.
3. **Respon:** Token JWT dikirim ke sisi client (Frontend) untuk disimpan dan digunakan sebagai kunci akses pada request berikutnya.

Endpoint Profile dan Sesi

```

me/route.ts
import { NextRequest, NextResponse } from "next/server";
import { prisma } from "@lib/prisma";
import { authenticateRequest } from "@lib/auth";

```



```

export async function GET(request: NextRequest) {
  try {
    const auth = await authenticateRequest(request);
    if (auth.error) return auth.error;

    const { user } = auth;

    const userData = await prisma.user.findUnique({
      where: { id: user.userId },
      select: {
        id: true, email: true, name: true, role: true, createdAt:
true,
        jobs: { orderBy: { createdAt: 'desc' } },
      },
    });

    if (!userData) return NextResponse.json({ error: "User not found" },
{ status: 404 });

    return NextResponse.json({ user: userData });
  } catch (error) {
    console.error("Profile error:", error);
    return NextResponse.json({ error: "Server Error" }, { status: 500
});
  }
}

```

Penjelasan Alur Kode: Endpoint ini digunakan untuk memvalidasi sesi pengguna yang sedang aktif (Session Persistence).

1. **Middleware Auth:** Fungsi memanggil `authenticateRequest` di baris pertama. Fungsi ini mengecek validitas Token JWT dari header. Jika token palsu atau kedaluwarsa, akses langsung ditolak (401 Unauthorized).
2. **Data Fetching:** Jika token valid, sistem mengambil detail data user dari database berdasarkan `userId` yang ada di dalam token.
3. **Relasi Data:** Selain profil diri, sistem juga mengambil daftar lowongan (jobs) yang pernah diposting oleh user tersebut, diurutkan dari yang terbaru (`createdAt: 'desc'`).

D. Implementasi Modul Lowongan Kerja (Jobs)

Endpoint Manajemen Data dan Pencarian

jobs/route.ts

```

import { NextRequest, NextResponse } from "next/server"
import { prisma } from "@/lib/prisma"
import { Prisma, JobType } from "@prisma/client"

```

```

// Ambil pekerjaan
export async function GET(request: NextRequest) {
  try {
    const { searchParams } = new URL(request.url)
    const type = searchParams.get("type")
    const location = searchParams.get("location")
    const search = searchParams.get("search")

    const where: Prisma.JobWhereInput = {}

    if (type) where.type = type as JobType

    if (location) {
      where.location = {
        contains: location,
        mode: "insensitive",
      }
    }

    if (search) {
      where.OR = [
        { title: { contains: search, mode: "insensitive" } },
        { company: { contains: search, mode: "insensitive" } },
        { description: { contains: search, mode: "insensitive" } },
      ]
    }

    const jobs = await prisma.job.findMany({
      where,
      include: {
        user: {
          select: {
            id: true,
            name: true,
            role: true,
          },
        },
      },
      orderBy: { createdAt: "desc" },
    })

    return NextResponse.json({ jobs })
  } catch {
    return NextResponse.json(
      { error: "Gagal mengambil data job" },
      { status: 500 }
    )
  }
}

```

```

    }
  }

  // Buat pekerjaan
  export async function POST(request: NextRequest) {
    try {
      const body = await request.json()
      const {
        title,
        company,
        location,
        type,
        salary,
        description,
        requirements,
        userId,
      } = body

      if (!title || !company || !location || !description || !requirements
|| !userId) {
        return NextResponse.json(
          { error: "Data tidak lengkap" },
          { status: 400 }
        )
      }

      const job = await prisma.job.create({
        data: {
          title,
          company,
          location,
          type: type || "FULLTIME",
          salary: salary || null,
          description,
          requirements,
          userId,
        },
      })

      return NextResponse.json(job, { status: 201 })
    } catch {
      return NextResponse.json(
        { error: "Gagal membuat lowongan" },
        { status: 500 }
      )
    }
  }
}

```

Penjelasan Alur Kode: File ini menangani dua fungsi utama:

1. **GET** (Pencarian & Filtering):

- Menggunakan URLSearchParams untuk menangkap parameter pencarian dari pengguna, yaitu type, location, dan search (kata kunci).
- Logika Query Dinamis: Sistem menyusun klausa where pada Prisma secara otomatis.
 - Jika ada search, sistem akan mencari kecocokan pada kolom title, company, ATAU description sekaligus (OR condition).
 - Filter location bersifat case-insensitive (huruf besar/kecil dianggap sama).
- Hasil data diurutkan berdasarkan createdAt menurun (terbaru paling atas).

2. **POST** (Pembuatan Lowongan):

- Menerima input data lowongan beserta userId dari frontend.
- Melakukan validasi kelengkapan data sebelum disimpan. Jika sukses, data disimpan dengan status default tipe pekerjaan "FULLTIME" jika tidak dipilih.

Endpoint Detail dan Modifikasi

jobs/[id]/route.ts

```
import { NextRequest, NextResponse } from "next/server"
import { prisma } from "@/lib/prisma"

// Helper: Ambil ID dari URL
function getId(request: NextRequest) {
  const segments = new
URL(request.url).pathname.split("/").filter(Boolean)
  return segments[segments.length - 1]
}

// Ambil pekerjaan berdasarkan ID
export async function GET(request: NextRequest) {
  try {
    const id = getId(request)
    if (!id) return NextResponse.json({ error: "ID tidak valid" }, {
status: 400 })

    const job = await prisma.job.findUnique({
      where: { id },
      include: { user: { select: { id: true, name: true, email: true,
role: true } } },
    })
  }
```

```

    if (!job) return NextResponse.json({ error: "Lowongan tidak
ditemukan" }, { status: 404 })

    return NextResponse.json(job, { status: 200 })
  } catch (error) {
    console.error("GET job by ID error:", error)
    return NextResponse.json({ error: "Terjadi kesalahan server" }, {
status: 500 })
  }
}

// UPDATE JOB (Hanya Owner/Admin)
export async function PUT(request: NextRequest) {
  try {
    const body = await request.json()
    const id = getId(request)
    const { userId, title, company, location, type, salary, description,
requirements } = body

    if (!id || !userId) return NextResponse.json({ error: "ID atau userId
tidak valid" }, { status: 400 })
    if (!title || !company || !location || !description || !requirements)
{
      return NextResponse.json({ error: "Data tidak lengkap" }, { status:
400 })
    }

    const existingJob = await prisma.job.findUnique({ where: { id } })
    if (!existingJob) return NextResponse.json({ error: "Lowongan tidak
ditemukan" }, { status: 404 })

    const isOwner = existingJob.userId === userId
    const isAdmin = body.role === "ADMIN"

    if (!isOwner && !isAdmin) return NextResponse.json({ error: "Anda
bukan pemilik lowongan ini" }, { status: 403 })

    const updatedJob = await prisma.job.update({
      where: { id },
      data: { title, company, location, type, salary, description,
requirements },
    })

    return NextResponse.json({ message: "Update sukses", job: updatedJob
}, { status: 200 })
  } catch (error) {
    console.error("PUT job error:", error)
  }
}

```

```

    return NextResponse.json({ error: "Terjadi kesalahan server" }, {
status: 500 })
  }
}

// DELETE JOB (Hanya Owner/Admin)
export async function DELETE(request: NextRequest) {
  try {
    const body = await request.json()
    const id = getId(request)
    const userId = body.userId
    const role = body.role

    if (!id || !userId) return NextResponse.json({ error: "ID atau userId
tidak valid" }, { status: 400 })

    const existingJob = await prisma.job.findUnique({ where: { id } })
    if (!existingJob) return NextResponse.json({ error: "Lowongan tidak
ditemukan" }, { status: 404 })

    const isOwner = existingJob.userId === userId
    const isAdmin = role === "ADMIN"

    if (!isOwner && !isAdmin) return NextResponse.json({ error: "Anda
bukan pemilik lowongan ini" }, { status: 403 })

    await prisma.job.delete({ where: { id } })

    return NextResponse.json({ message: "Lowongan berhasil dihapus" }, {
status: 200 })
  } catch (error) {
    console.error("DELETE job error:", error)
    return NextResponse.json({ error: "Terjadi kesalahan server" }, {
status: 500 })
  }
}

```

Penjelasan Alur Kode: File ini menangani operasi spesifik berdasarkan ID lowongan:

1. **Helper getId:** Fungsi kecil untuk memarsing ID lowongan dari URL request secara aman.
2. **GET (Detail):** Mengambil satu data lowongan lengkap dengan relasi profil user pembuatnya (include: { user: ... }).
3. **PUT & DELETE** (Keamanan Bertingkat):

- Sebelum mengizinkan Edit atau Hapus, sistem melakukan pengecekan ketat.
- **Validasi Role:** Sistem mengecek apakah user yang request adalah Pemilik Asli (`existingJob.userId === userId`) ATAU seorang **ADMIN** (`role === "ADMIN"`).
- **Authorization Guard:** Jika bukan pemilik dan bukan admin, sistem menolak akses dengan status 403 Forbidden. Ini mencegah manipulasi data ilegal.